

exp 2a sentiment analysis explanation

13 May 2026 01:56

DL Assignment 2 — Sentiment Analysis on IMDB Reviews using Deep Learning

This assignment performs:

Sentiment Analysis using Deep Neural Network

using:

- TensorFlow
- TensorFlow Hub
- IMDB movie reviews dataset
- pretrained word embeddings

1. What We Have To Do?

We have to:

1. load IMDB movie review dataset
2. preprocess text reviews
3. convert text into embeddings
4. build deep learning classification model
5. train model
6. classify reviews as:
 - positive
 - negative
7. evaluate performance

Main Goal

To predict:

sentiment of movie reviews

Example

Review	Sentiment
"Amazing movie"	Positive
"Worst film ever"	Negative

2. What Type of Problem is This?

This is:

Binary Classification Problem

Because output has only:

2 classes

Label	Meaning
0	Negative
1	Positive

3. Overall Workflow

Load Dataset



Convert Text to Embeddings



Build Neural Network



Train Model



Predict Sentiment



Evaluate Accuracy

CELL 1 — Install Libraries

```
!pip install tensorflow tensorflow-hub tensorflow-datasets
```

Purpose

Installs required libraries.

Libraries Used

Library	Purpose
TensorFlow	deep learning
TensorFlow Hub	pretrained embeddings
TensorFlow Datasets	IMDB dataset

CELL 2 — Import Libraries

```
import tensorflow as tf
import tensorflow_hub as hub
import tensorflow_datasets as tfds
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import metrics
```

Line-by-Line

TensorFlow

```
import tensorflow as tf
Used for:
```

building neural network

TensorFlow Hub

```
import tensorflow_hub as hub
Used for:
```

pretrained embeddings

TensorFlow Datasets

import tensorflow_datasets as tfds
Loads IMDB dataset.

pandas

import pandas as pd
Used for graph plotting.

numpy

import numpy as np
Numerical operations.

matplotlib

import matplotlib.pyplot as plt
Graph visualization.

metrics

from sklearn import metrics
Used for:

- classification report
- confusion matrix

CELL 3 — Load Dataset

```
train_data, validation_data, test_data = tfds.load(  
    name='imdb_reviews',  
    split=('train[:60%]', 'train[60%:]', 'test'),  
    as_supervised=True  
)
```

What is IMDB Dataset?

Dataset containing:

- movie reviews
- sentiment labels

Dataset Split

Dataset	Purpose
train_data	learning
validation_data	checking during training
test_data	final testing

train[:60%]

Uses:

first 60% reviews for training

train[60%:]

Remaining training data used for:

validation

as_supervised=True

Returns:

(review, label)

pairs.

CELL 4 — View Sample Batch

```
train_examples_batch, train_labels_batch = next(iter(train_data.batch(10)))  
print(train_labels_batch)
```

Purpose

Extracts:

first 10 reviews and labels

batch(10)

Creates batch of:

10 samples

iter()

Creates iterator.

next()

Fetches first batch.

Labels

Value	Meaning
0	Negative
1	Positive

CELL 5 — Pretrained Embedding

embedding = "<https://tfhub.dev/google/nnlm-en-dim128-with-normalization/2>"

What is Embedding?

Embedding converts:

text → numerical vectors

because neural networks understand only numbers.

Pretrained Embedding Means

Embedding already trained by:

Google

on huge text datasets.

dim128

Each review converted into:

128-dimensional vector

CELL 6 — Custom HubLayer

```
class HubLayer(tf.keras.layers.Layer):
```

Purpose

Wraps TensorFlow Hub embedding into:

Keras-compatible layer

Why Needed?

Allows embedding to work inside:
Sequential model

CELL 7 — Input Layer

```
inputs = tf.keras.Input(shape=(), dtype=tf.string)
```

Purpose

Defines:

text input layer

dtype=tf.string

Input type is:
text/string

shape=()

Single sentence/review input.

CELL 8 — Embedding Layer

```
x = HubLayer(embedding)(inputs)
```

Purpose

Converts text reviews into:

numerical embeddings

Example

Review:

"movie was amazing"

becomes vector:

[0.21, -0.8, ...]

CELL 9 — Hidden Layers

```
x = tf.keras.layers.Dense(32, activation='relu')(x)
```

```
x = tf.keras.layers.Dense(16, activation='relu')(x)
```

Purpose

Learns:

- positive patterns
- negative patterns
- semantic relationships

ReLU Activation

Formula:

$f(x) = \max(0, x)$

Why ReLU Used?

Advantages:

- fast
- avoids vanishing gradient
- efficient training

CELL 10 — Output Layer

```
outputs = tf.keras.layers.Dense(1)(x)
```

Purpose

One neuron because:

binary classification

Output

Produces:

logits/raw prediction values

CELL 11 — Build Model

```
model = tf.keras.Model(inputs, outputs)
```

Purpose

Creates complete neural network model.

Model Architecture

Text Input



Embedding Layer



Dense(32)



Dense(16)



Output(1)

CELL 12 — Model Summary

`model.summary()`

Purpose

Displays:

- layers
- output shapes
- parameters

CELL 13 — Compile Model

```
model.compile(  
    optimizer='adam',  
    loss=tf.keras.losses.BinaryCrossentropy(from_logits=True),  
    metrics=['accuracy']  
)
```

optimizer='adam'

Controls:

weight updates

BinaryCrossentropy

Used for:

binary classification

Formula:

$L = -\left(y \log(\hat{y}) + (1-y) \log(1-\hat{y})\right)$

from_logits=True

MOST IMPORTANT.

Output layer has:

`Dense(1)`

without sigmoid.

So loss function internally applies sigmoid.

metrics=['accuracy']

Measures:

correct prediction percentage

CELL 14 — Train Model

```
history = model.fit(  
    train_data.shuffle(10000).batch(512),  
    epochs=5,  
    validation_data=validation_data.batch(512),  
    verbose=1  
)
```

shuffle(10000)

Randomly mixes reviews.

Improves generalization.

batch(512)

Processes:
512 reviews together

epochs=5

Complete dataset processed:
5 times

validation_data

Checks model performance during training.

history Variable

Stores:

- loss
- accuracy
- validation metrics

CELL 15 — Evaluate Model

```
results = model.evaluate(test_data.batch(512))
```

Purpose

Tests model on:

unseen reviews

Output Example

Accuracy = 0.79

Meaning:

79% predictions correct

CELL 16 — Plot Training Graph

```
pd.DataFrame(history.history).plot(figsize=(10,7))  
plt.show()
```

Purpose

Plots:

- accuracy
- loss
- validation metrics

Good Graph

accuracy ↑

loss ↓

CELL 17 — Convert Test Reviews


```
texts = []
true_labels = []
for text, label in test_data.take(200):
    texts.append(text.numpy().decode('utf-8'))
    true_labels.append(label.numpy())
```

Purpose

Extracts:

- review texts
- labels

decode('utf-8')

Converts:

bytes → string

CELL 18 — Predictions

```
texts_tensor = tf.constant(texts)
predicted_probs = model.predict(texts_tensor)
predicted_labels = (predicted_probs > 0).astype(int)
```

model.predict()

Predicts sentiment logits.

(predicted_probs > 0)

If prediction:

> 0

→ Positive

Else:

→ Negative

Why Threshold 0?

Because:

from_logits=True

Logits centered around 0.

CELL 19 — Classification Report

```
print(metrics.classification_report(true_labels, predicted_labels))
```

Metrics Explained

Metric	Meaning
--------	---------

Precision	correctness of positive predictions
-----------	-------------------------------------

Recall	ability to find positives
--------	---------------------------

F1-score	balance of precision and recall
----------	---------------------------------

CELL 20 — Confusion Matrix

```
cm = metrics.confusion_matrix(true_labels, predicted_labels)
print(cm)
```

Confusion Matrix Meaning

Actual	Predicted	Meaning
Negative	Negative	correct
Negative	Positive	false positive
Positive	Positive	correct
Positive	Negative	false negative

Important Concepts Used

Concept	Purpose
NLP	text processing
Embeddings	text → vectors
Binary Classification	positive/negative
Deep Learning	sentiment learning

Important Viva Questions

Q1. What is sentiment analysis?

Classifying text as positive or negative.

Q2. Why embeddings used?

Neural networks cannot understand raw text.

Q3. What are pretrained embeddings?

Embeddings already trained on huge text datasets.

Q4. Why BinaryCrossentropy used?

Because this is binary classification.

Q5. What is from_logits=True?

Indicates output layer produces raw logits.

Q6. Why ReLU used?

Fast and efficient activation function.

Q7. What is confusion matrix?

Table showing correct and incorrect predictions.

Q8. What is precision?

Correctness of positive predictions.

Q9. What is recall?

Ability to identify actual positives.

Short Practical Explanation

This program performs sentiment analysis on IMDB movie reviews using TensorFlow and TensorFlow Hub pretrained embeddings. Text reviews are converted into numerical vectors and classified as positive or negative using a deep neural network. The model is evaluated using accuracy, classification report, and confusion matrix.